

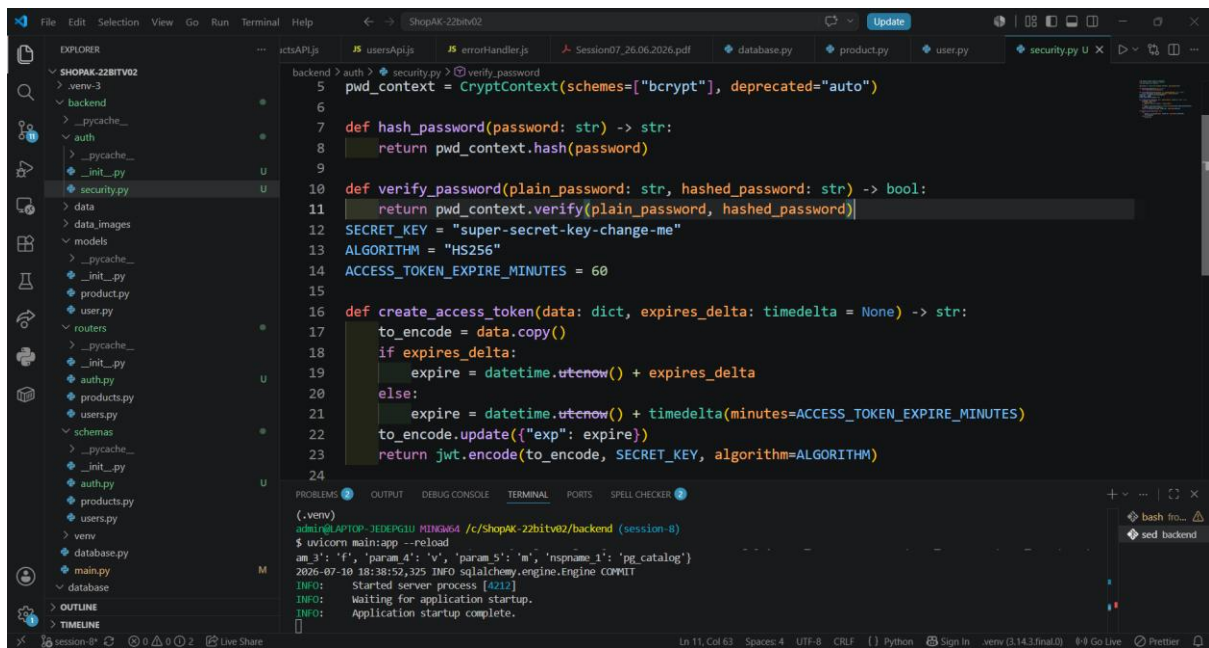
Session09_02.07.2026

In-class Lab Tasks

1. Backend: Password Hashing & JWT

Implement **hash_password** and **verify_password** using Passlib.

Implement **create_access_token** using python-jose.



The screenshot shows a VS Code editor with a project named 'ShopAK-22bitv02'. The file explorer on the left shows a directory structure with files like 'security.py', 'user.py', 'product.py', 'database.py', 'main.py', 'schemas.py', 'users.py', 'products.py', 'auth.py', 'init.py', 'data.py', 'data_images.py', 'models.py', 'pycache_', and 'routes'. The main editor displays the 'security.py' file with the following code:

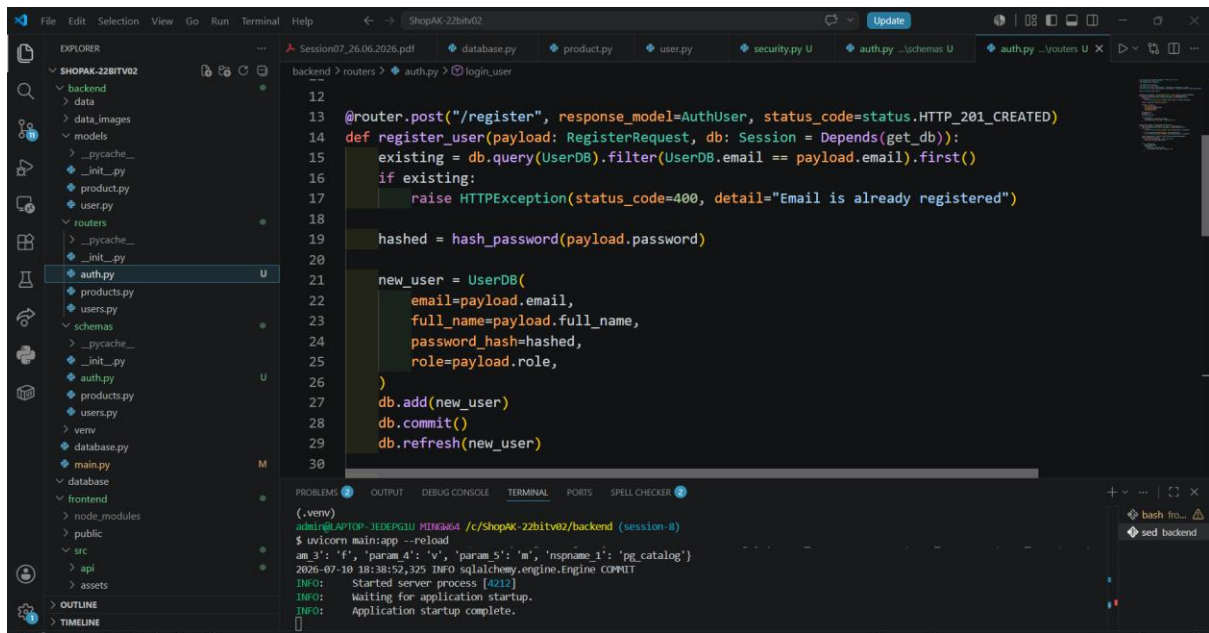
```
5 pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
6
7 def hash_password(password: str) -> str:
8     return pwd_context.hash(password)
9
10 def verify_password(plain_password: str, hashed_password: str) -> bool:
11     return pwd_context.verify(plain_password, hashed_password)
12
13 SECRET_KEY = "super-secret-key-change-me"
14 ALGORITHM = "HS256"
15 ACCESS_TOKEN_EXPIRE_MINUTES = 60
16
17 def create_access_token(data: dict, expires_delta: timedelta = None) -> str:
18     to_encode = data.copy()
19     if expires_delta:
20         expire = datetime.utcnow() + expires_delta
21     else:
22         expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
23     to_encode.update({"exp": expire})
24     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

The bottom panel shows the terminal output:

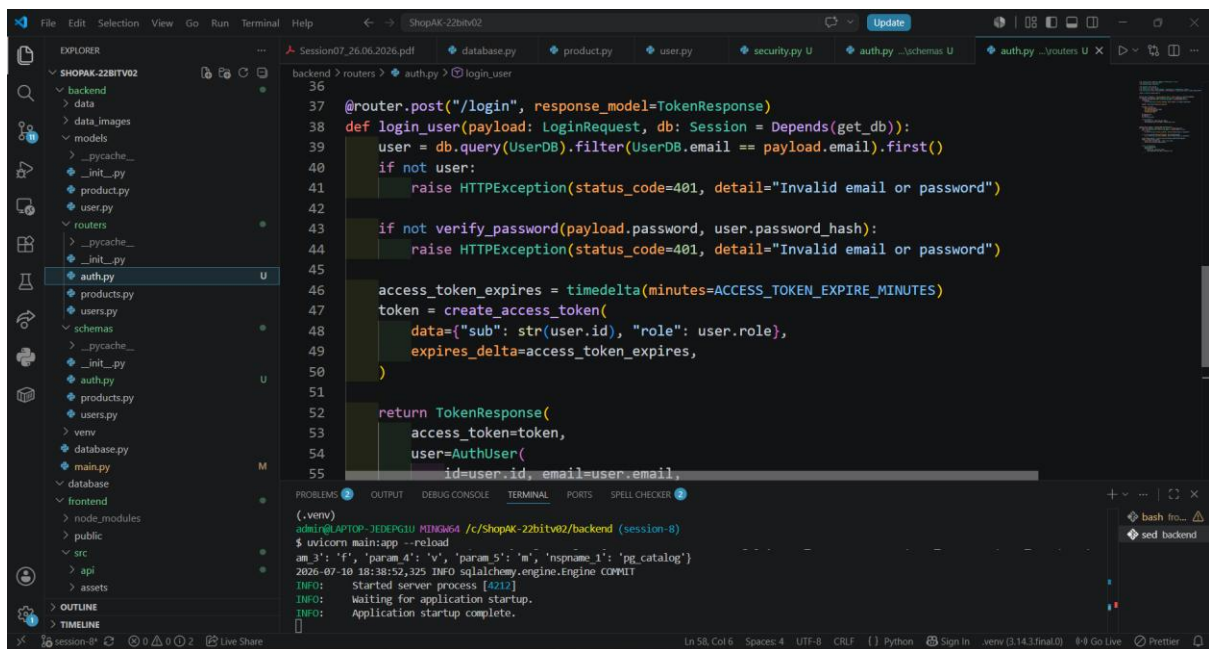
```
(.venv)
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02/backend (session-8)
$ uvicorn main:app --reload
INFO: Started server process [4212]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Implement POST /register and POST /login with proper validation and error handling.

POST/register



POST/login



2. Backend: Swagger Testing

a. Test Register:

Khi nhập vào thông tin không có (password/password_hash) trong response chỉ có (id, email, full_name, role)

auth

POST /register Register User

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{
  "email": "long@gmail.com",
  "full_name": "User",
  "password": "User123",
  "role": "User"
}
```

Execute Clear

Responses

Bấm excute khi thành công sẽ trả về response 201

curl

```
curl -X 'POST' \
  'http://localhost:8000/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "long@gmail.com",
    "full_name": "User",
    "password": "User123",
    "role": "User"
  }'
```

Request URL

http://localhost:8000/register

Server response

Code Details

201

Response body

```
{
  "id": 4,
  "email": "long@gmail.com",
  "full_name": "User",
  "role": "User"
}
```

Response headers

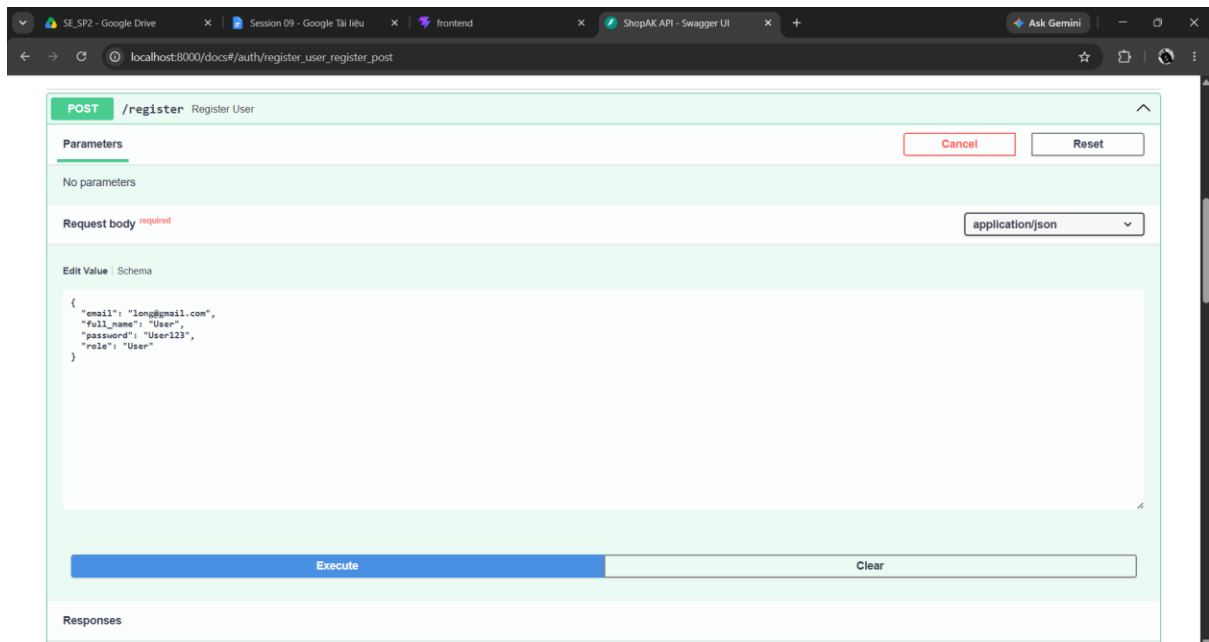
```
access-control-allow-credentials: true
content-length: 66
content-type: application/json
date: Fri, 10 Jul 2026 11:46:02 GMT
server: uvicorn
```

Responses

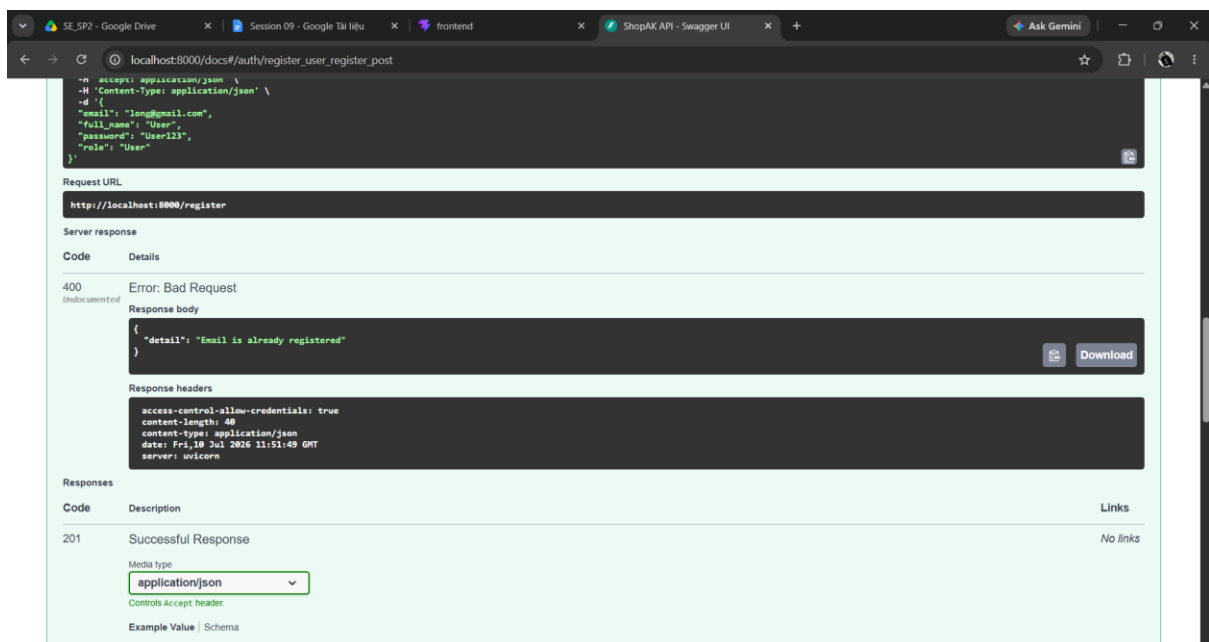
Code	Description	Links
201	Successful Response	No links

b. Test Login

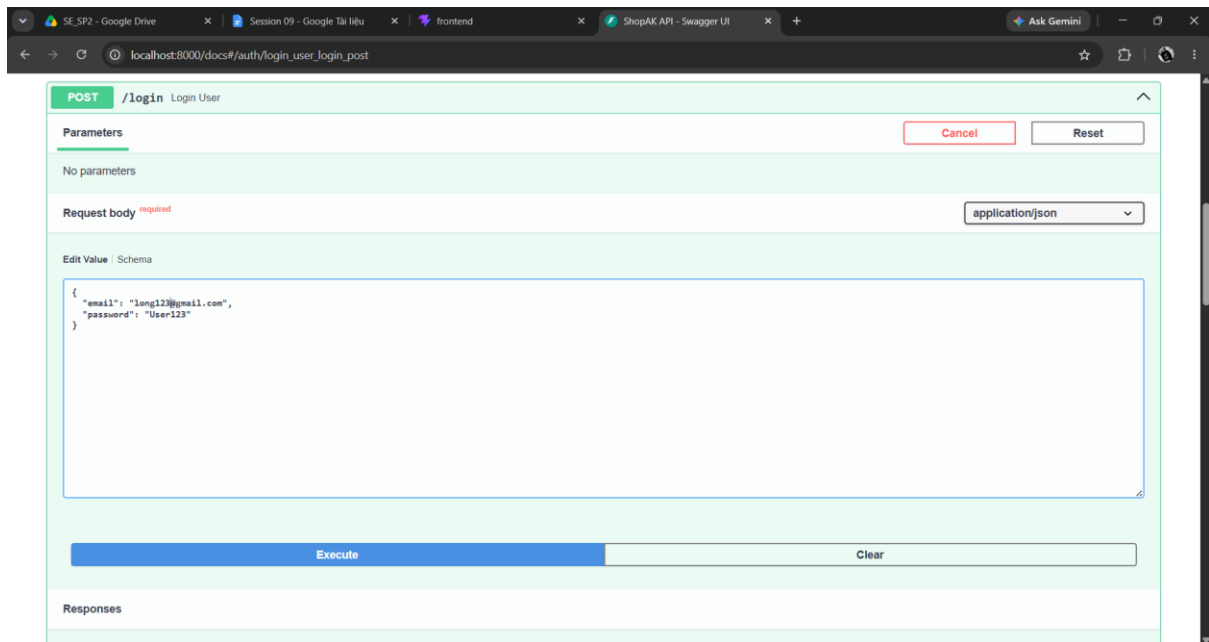
Nhập thông tin bên trong POST /login với thông tin đã đăng ký



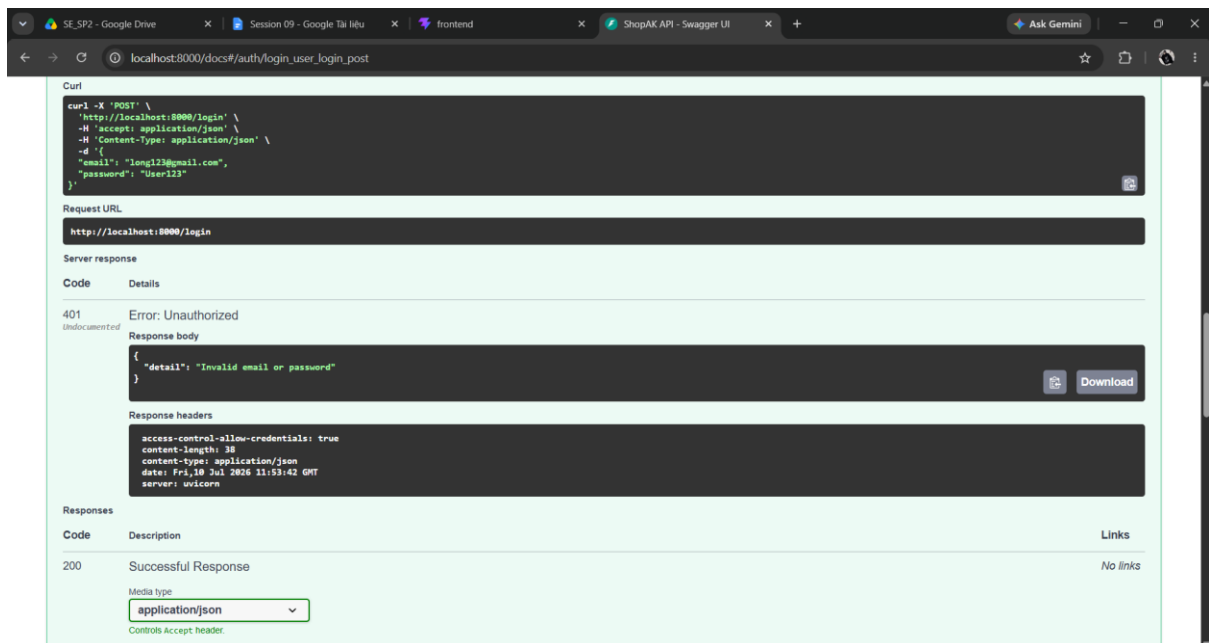
Khi cố gắng tạo tài khoản trùng lặp nó sẽ trả kết quả cho chúng ta là 400 với tài khoản email đã được đăng ký



Tương tự với phần login chúng ta sẽ test sai tài khoản và mật khẩu



Khi cô nhập sai thông tin tài khoản và mật khẩu không được đăng ký sẽ trả về kết quả 401: “Invalid email or password”



d. Verify trong sql DBBeaver

Để xác thực tính khả thi thì chúng ta sẽ test bên trong sql app

	A-Z email	A-Z full_name	A-Z password_hash	A-Z role
1	loivo@gmail.com	Võ Thành Lợi	123456	Admin
2	loi@gmail.com	Admin	\$2b\$12\$4taRjhonjf/2kvC1W	Admin
3	loi123@gmail.com	loi	\$2b\$12\$VHjY03i2W0CuC9iY	customer
4	long@gmail.com	User	\$2b\$12\$ka7UsrKYHozFK9As	User

Chúng ta sẽ thấy user vừa tạo, cột password_hash là chuỗi hash dài (bắt đầu bằng \$2b\$...), không phải plaintext 123456 tài khoản loivo@gmail.com đã được tạo trước khi thêm password_hash tính bảo mật của tài khoản đã được thêm vô và tạo mới các tài khoản như trên ảnh.

3. Frontend: Registration Page

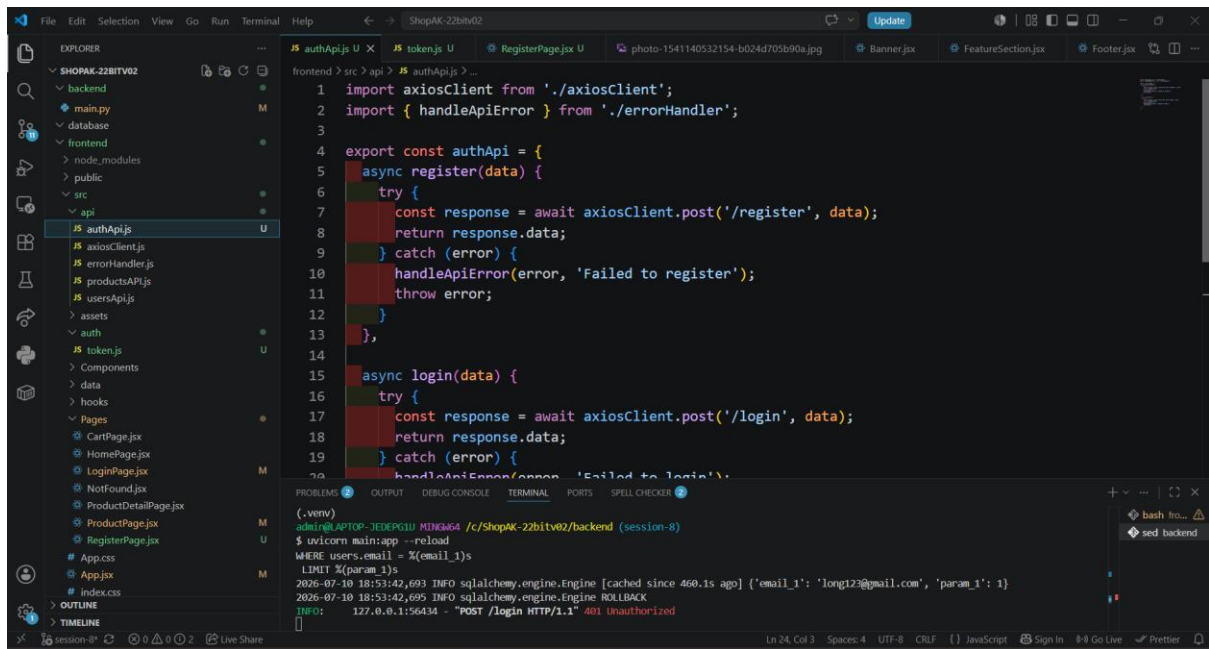
Trang register page:

```

1 import { useState } from 'react';
2 import { Link, useNavigate } from 'react-router-dom';
3 import { authApi } from '../api/authApi';
4
5 const RegisterPage = () => {
6   const [form, setForm] = useState({ email: '', fullName: '', password: '' });
7   const [loading, setLoading] = useState(false);
8   const [error, setError] = useState('');
9   const [successMessage, setSuccessMessage] = useState('');
10  const navigate = useNavigate();
11
12  const handleChange = (e) => {
13    setForm((prev) => ({ ...prev, [e.target.name]: e.target.value }));
14  };
15
16  const handleSubmit = async (e) => {
17    e.preventDefault();
18    setLoading(true);
19    setError('');
20
21    // ... registration logic ...
22  };
23
24  return (
25    <div>
26      <h2>Register</h2>
27      <form>
28        <input type="text" value={form.fullName} />
29        <input type="text" value={form.email} />
30        <input type="password" value={form.password} />
31        <button type="submit">Register</button>
32      </form>
33    </div>
34  );
35 }

```

authApi.js:



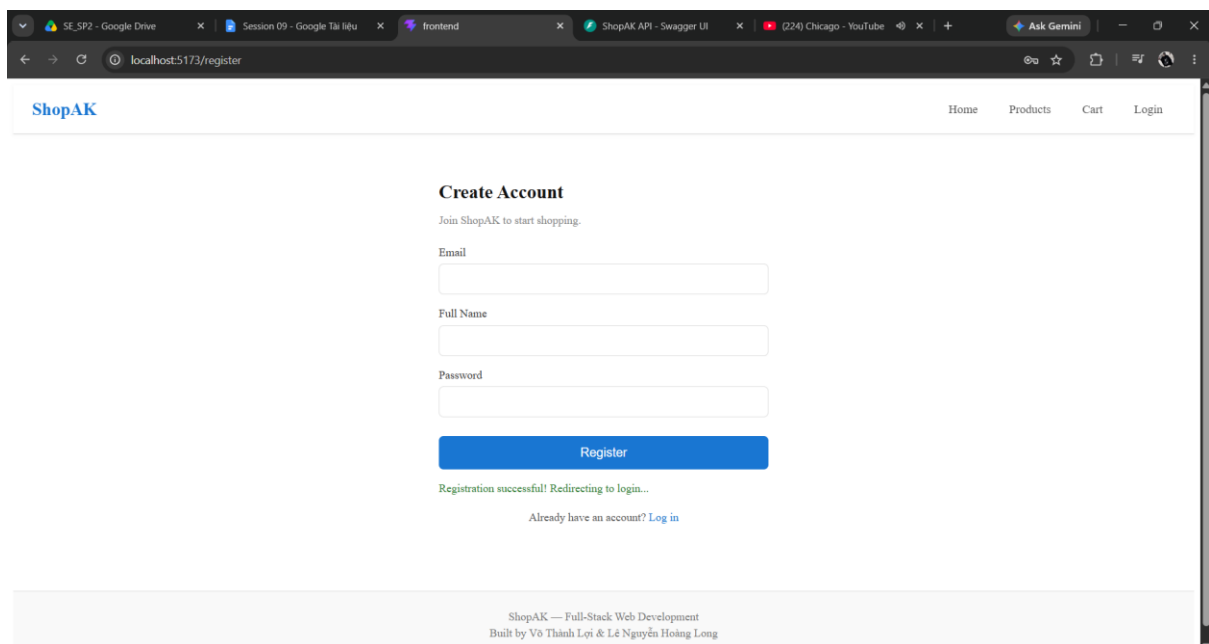
The screenshot shows a VS Code editor with the file explorer on the left displaying the project structure. The main editor shows the `authApi.js` file with the following code:

```
1 import axiosClient from './axiosClient';
2 import { handleApiError } from './errorHandler';
3
4 export const authApi = {
5   async register(data) {
6     try {
7       const response = await axiosClient.post('/register', data);
8       return response.data;
9     } catch (error) {
10      handleApiError(error, 'Failed to register');
11      throw error;
12    }
13  },
14  async login(data) {
15    try {
16      const response = await axiosClient.post('/login', data);
17      return response.data;
18    } catch (error) {
19      handleApiError(error, 'Failed to login');
20      throw error;
21    }
22  }
23 }
```

The terminal window at the bottom shows the following output:

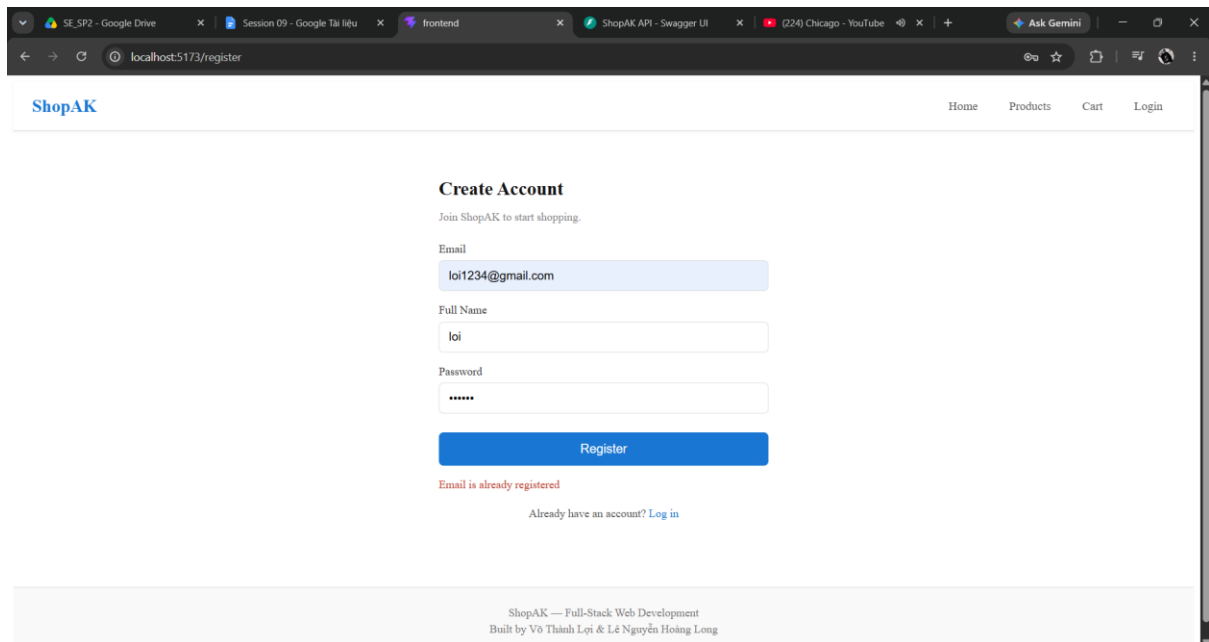
```
(.env)
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02/backend (session-8)
$ uvicorn main:app --reload
INFO: users.email = %(email_1)s
LIMIT %(param_1)s
2026-07-10 18:53:42,693 INFO sqlalchemy.engine.Engine [cached since 460.1s ago] ('email_1': 'long123@gmail.com', 'param_1': 1)
2026-07-10 18:53:42,695 INFO sqlalchemy.engine.Engine ROLLBACK
INFO: 127.0.0.1:56434 - "POST /login HTTP/1.1" 401 unauthorized
```

Chúng ta sẽ test phần register sau khi điền form và submit thành công sẽ hiện message: “Registration Sucessful”



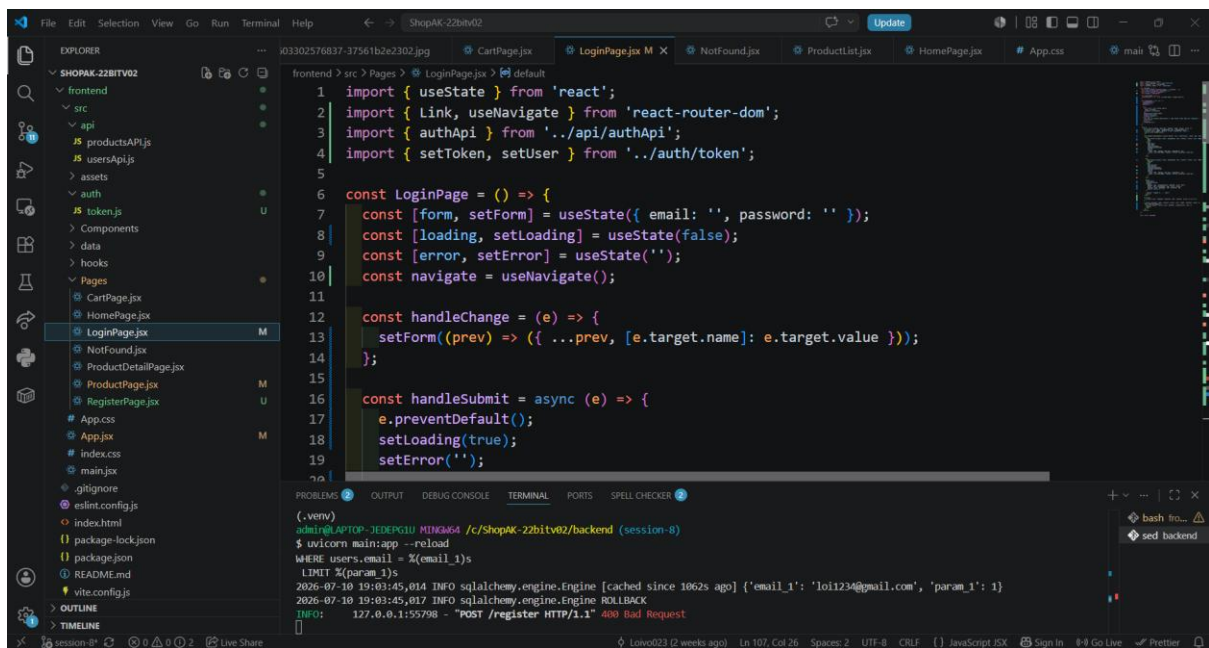
Tiếp theo chúng ta sẽ test thử với tài khoản đã được đăng ký để kiểm chứng tính bảo mật cũng như tránh trùng tài khoản

Sau khi nhập sẽ trả về kết quả: “Email is already registered”

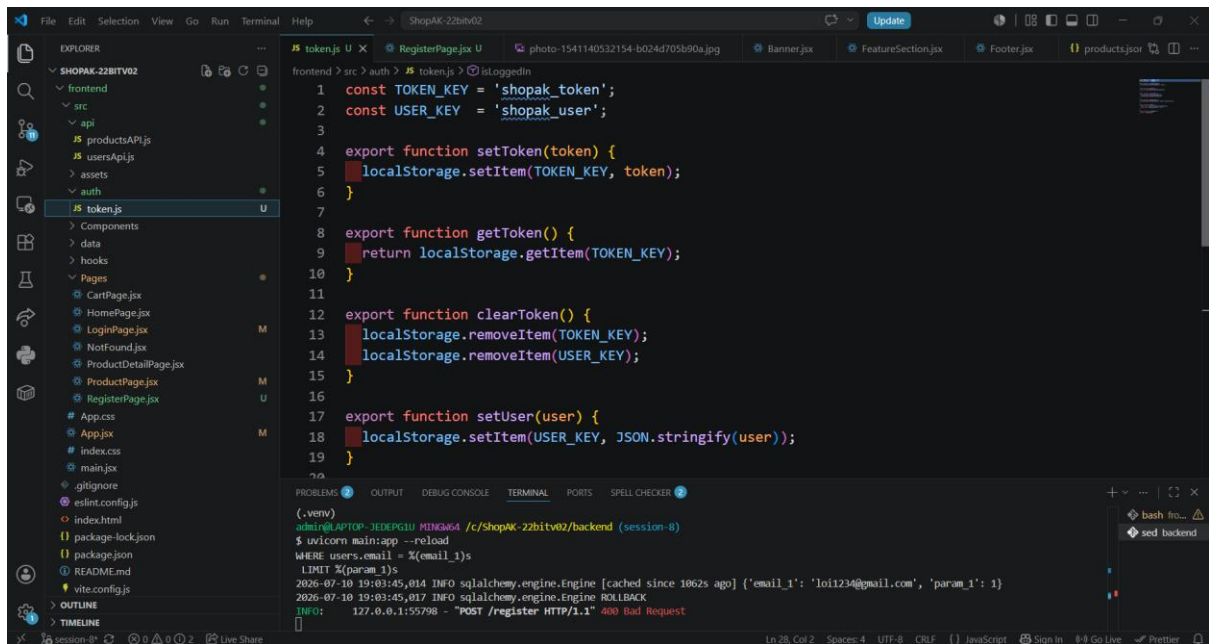


4. Frontend: Login Page

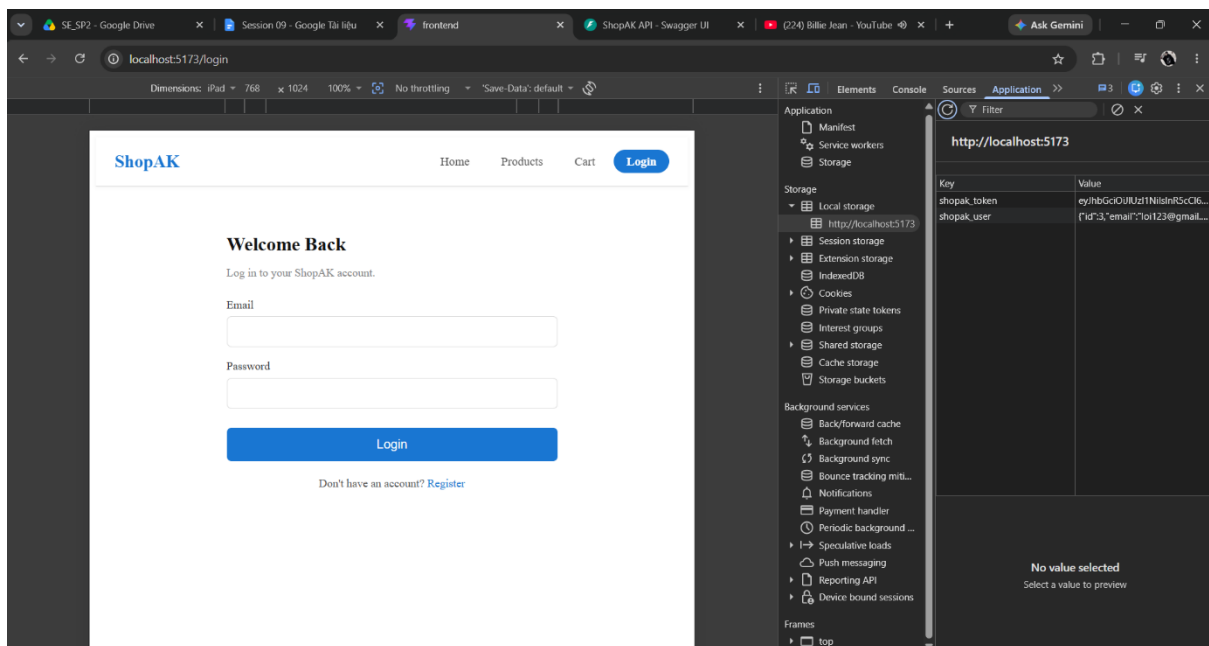
Code trang Loginpage.jsx



Code token với hàm setToken



Khi mở F12 ta sẽ thấy shopak_token đã được lưu với giá trị JWT dài



sau khi login đã tự động chuyển sang /products xác nhận redirect hoạt động

